Understanding LangChain Document Loaders

1. Introduction

A document loader reads data from a source, such as a file, a web page or a database, and turns it into a standard format that an LLM application can use. In LangChain this format is the **Document** object, which holds two things: the text itself (*page_content*) and information about where it came from (*metadata*), for example the file name or the page number.

We need document loaders because LLMs do not know our private data. Company manuals, contracts, policies or emails must first be loaded, then split into chunks, turned into embeddings and stored in a vector database. Loading is the first step of every RAG (Retrieval-Augmented Generation) pipeline: if the text is loaded badly, every step after it gets worse.

In this demo we use **LangChain**, an open-source framework that gives one common interface for loaders, text splitters, embeddings, vector stores and LLMs (chat models from OpenAI, Anthropic, Google or local open-source models). Besides loaders, it offers prompt templates for structured inputs, output parsers for structured outputs, chains for sequential steps, agents for autonomous decisions, memory for conversation context and vector stores for semantic search.

2. Types of Document Loaders

The most common loaders, and how many Documents each one creates:

TextLoader:	Load plain text files (.txt) (one Document per file)
CSVLoader:	Load CSV files with row-based documents (one Document per row)
JSONLoader:	Load JSON files with jq-style extraction (one Document per extracted record)
UnstructuredLoader:	Handle various file formats automatically, e.g. Word, PowerPoint, HTML (one Document per file, or one per element such as title or paragraph)
DirectoryLoader:	Load all files from a directory with a chosen loader (gives what that loader gives, e.g. one Document per page with PyPDFLoader)
WebBaseLoader:	Scrape and load web pages (one Document per URL)
YoutubeLoader:	Extract transcripts from YouTube videos (one Document per video)

For PDF files there are three common choices, all giving one Document per page:

PyPDFLoader:	The simplest option. Pure Python, easy to install, good for clean text-based PDFs.
PyMuPDFLoader:	Faster and usually better text extraction, with richer metadata. Good default for larger files.
PDFPlumberLoader:	Slower, but keeps layout details. Best when the PDF has tables or complex columns.

3. Code Example: Loading a PDF

The simplest way to load a PDF with PyPDFLoader (install with *pip install pypdf*):

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("docs/langchain_document_loaders.pdf")
documents = loader.load()  # one Document per page

print(f"Loaded {len(documents)} pages")
print(documents[0].page_content[:200])
print(documents[0].metadata)
```

Each page becomes its own Document, and the metadata keeps the source file and page number.

4. Best Practices

- **Check the output:** always print a sample of the loaded text. Scanned PDFs, tables and columns often load badly.
- **Keep good metadata:** source, page and date help with filtering and with citing answers.
- **Clean the text:** remove repeated headers, footers and extra white space before splitting.
- **Pick the right loader:** test two or three loaders on your real files and keep the best one.
- **Handle errors:** one broken file should not stop the whole batch.

5. Conclusion

Document loaders are the entry point of a RAG system. They turn files and other sources into Documents with text and metadata, ready for splitting and retrieval. Good loading leads to better chunks, better search and better answers.